

SOLARI

SOLAR AI CHATBOT

KNOWLEDGE BASE DOCUMENTATION

Prepared by	Muhammad Ahmad Hassan
Project	Solari — Solar AI Chatbot
Assigned Track	Knowledge Base Documentation
Document Type	Project Technical Documentation
Version	1.0
Date	August 2026

A professional, implementation-based record of how Solari's solar knowledge is organized, loaded, grounded, protected by safety rules, and connected to deterministic sizing calculations.

Executive Summary

Solari uses a curated knowledge layer to provide the AI assistant with reliable solar-energy information. The runtime knowledge base is organized into three Markdown files covering hardware, systems, and operations/support. At application startup, these files are loaded and combined into a single in-memory knowledge base. The system prompt then instructs Gemini to use this material as its primary factual source.

The project also includes three PDF research documents that are exposed through the frontend as reference material. These PDFs are not the files directly loaded by the runtime knowledge loader. The Markdown files form the AI's runtime context, while the PDFs provide user-visible supporting documentation.

Numerical solar recommendations are handled separately through a deterministic Python function. This gives the project a clear boundary between factual knowledge, conversational reasoning, and engineering calculations.

1. Knowledge Base Architecture

The knowledge base is intentionally simple and maintainable. Instead of using a vector database or a complex retrieval pipeline, the supplied project stores domain knowledge as Markdown and loads the complete content during backend startup.

Layer	Implementation	Purpose
Research content	3 Markdown files	Runtime factual knowledge
Loader	load_knowledge_base()	Reads and combines source files
Runtime state	app.state.knowledge_base	Stores combined knowledge
Behavior rules	system_prompt.txt	Grounding, safety, response rules
AI model	Gemini	Generates conversational answers
Calculation layer	calculate_solar_recommendation()	Deterministic preliminary sizing
Reference layer	3 PDF documents	User-visible supporting research

2. Runtime Knowledge Sources

The backend/data directory contains three Markdown files. Together they form the runtime knowledge base.

File	Primary coverage
hardware_research.md	PV modules, electrical specifications, inverter technology, MPPT, batteries, BMS, hardware comparisons and selection guidance.
operations_support.md	Installation, maintenance, warranties, pricing/economics, customer support, operational guidance and safety-related handling.
systems_research.md	On-grid, off-grid and hybrid architectures, system components, batteries, grid interaction, Pakistan regulations, design methodology and case studies.

These files collectively cover the project's main factual domains: hardware, system architecture, installation and operations, maintenance, safety, economics, and Pakistan-specific solar context.

3. Supporting PDF Research

The project also contains three PDF documents in backend/knowledge_base/. They are mounted as static reference documents and linked from the frontend Knowledge Base section.

PDF	Pages	Purpose
Pakistan Solar Operations Research.pdf	17	Pakistan-focused operations and practical solar research.
Solar_Hardware_Research.pdf	27	Solar hardware and component research.
Solar_PV_Systems_Report.pdf	31	PV system architecture and technical research.

Key distinction: these PDFs are reference documents for users. The backend loader directly reads the three Markdown files when constructing the AI knowledge base.

4. How the Knowledge Base Is Loaded

At FastAPI startup, the project's lifespan logic calls `load_knowledge_base()`. The loader checks for the three expected files, reads their UTF-8 text, strips the content, and joins the available sections with separators.

```
expected_files = [
    "hardware_research.md",
    "operations_support.md",
    "systems_research.md"
]

for file_name in expected_files:
    file_path = base_dir / file_name
    if file_path.exists():
        content = file_path.read_text(encoding="utf-8")
        knowledge_parts.append(content.strip())

return "\n\n---\n\n".join(knowledge_parts).strip()
```

The resulting text is stored in `app.state.knowledge_base`. Because the source is loaded at startup, updates to the Markdown files require the backend to be restarted before the new information is used.

5. Grounding Gemini With the Knowledge Base

The combined knowledge base is injected into the model's system instruction together with the project's system prompt and the selected solar system type. This gives Gemini both the factual material and the behavioral rules required to answer consistently.

- **Primary factual source:** use the supplied knowledge base for technical facts, specifications, regulations and operating guidance.
- **No fabrication:** do not invent missing specifications, prices, policies or performance guarantees.
- **Transparent uncertainty:** if the requested information is absent, acknowledge the limitation rather than guessing.
- **Domain boundary:** questions unrelated to solar energy, power electronics, electrical backup or utility billing are treated as out of scope.
- **Technical precision:** distinguish power (kW) from energy (kWh), and nameplate ratings from real-world output.
- **Readable output:** use headings, lists, bold terminology and comparison tables when useful.

6. Pakistan-Specific Knowledge

A major strength of the knowledge base is its localization for Pakistan. The system prompt and research content address local grid-connected solar considerations instead of relying only on generic international guidance.

- **Grid connection:** interconnection, sanctioned load, three-phase requirements and related technical considerations.

- **Net-billing context:** the project emphasizes the distinction between current net-billing arrangements and older 1:1 net-metering assumptions.
- **Local economics:** self-consumption, export compensation, CAPEX/OPEX and payback considerations are part of the operational knowledge.
- **Installation practice:** site assessment, structural checks, electrical protection and professional involvement are emphasized.

Regulatory information is time-sensitive. Future maintenance should review the regulatory sections of the knowledge files and system prompt together before deployment.

7. Safety and Professional Guardrails

Safety rules are embedded into the assistant's behavior. The system prompt prevents the chatbot from becoming a source of unsafe electrical instructions.

- **No zero-bill guarantees:** the assistant must not promise a completely zero electricity bill.
- **No live-electrical DIY procedures:** it must not provide step-by-step guidance for live DC wiring, live MC4 work, opening inverter chassis, live string testing or bypassing internal protection.
- **Fault escalation:** smoke, burning smells, sparking and persistent serious faults require professional safety escalation.
- **Panel-cleaning safety:** the prompt discourages unsafe cleaning practices during peak heat/sun conditions.
- **Engineering disclaimer:** preliminary sizing does not replace a professional site survey, shading analysis, structural assessment or final electrical design.

8. Knowledge vs. Calculation Engine

The project separates factual knowledge from numerical sizing. The function `calculate_solar_recommendation()` in `backend/utlis.py` performs deterministic calculations when the assistant needs a preliminary solar recommendation.

Parameter	Value	Purpose
Peak sun hours	4.5	Preliminary PV generation estimate
Performance ratio	0.80	Accounts for system losses/performance
Panel rating	550 W	Panel-count calculation
Inverter efficiency	0.95	Battery estimate
Battery DoD	0.80	Usable battery-energy assumption
Minimum system size	3.0 kW	Minimum recommended size
Panel area	30 ft ²	Preliminary roof-area estimate

Using a deterministic tool for sizing improves repeatability and reduces the risk of inconsistent arithmetic in free-form model responses.

9. End-to-End Knowledge Flow

01	Research	Domain knowledge is maintained in Markdown.
02	Load	FastAPI startup loads the three files.

03	Store	Combined text is kept in <code>app.state.knowledge_base</code> .
04	Ground	System prompt combines rules, knowledge and system type.
05	Generate	Gemini produces the grounded answer.
06	Calculate	Sizing requests can invoke the deterministic Python tool.
07	Explain	Gemini presents the result in natural language.

10. Maintenance Procedure

The knowledge base should be treated as a controlled project asset. A good update cycle is:

- Identify whether the change belongs to hardware, systems, operations/support or regulation.
- Update the relevant Markdown source with clear headings and verifiable information.
- If the change comes from a user-visible research document, update the corresponding PDF as well.
- Review `system_prompt.txt` when the change affects safety, regulations, required inputs or response behavior.
- Restart the backend so the updated knowledge is loaded.
- Run the project tests and verify both grounded answers and deterministic sizing behavior.

11. Strengths and Limitations

Strengths

- Clear separation of hardware, systems and operations knowledge.
- Pakistan-focused solar and grid context.
- Explicit grounding and anti-hallucination rules.
- Safety and professional escalation integrated into the AI behavior.
- Deterministic numerical sizing separated from language generation.
- User-visible PDF references improve transparency.

Limitations

- The current implementation loads the complete Markdown knowledge base rather than retrieving only the most relevant chunks.
- There is no vector database or semantic retrieval layer in the supplied implementation.
- Regulatory and pricing information can become outdated and therefore needs periodic review.
- Preliminary sizing remains an estimate and is not a replacement for professional engineering design.

12. Conclusion

Solari's knowledge architecture is a practical combination of curated Markdown research, explicit AI grounding rules, user-visible reference PDFs and deterministic solar-sizing calculations. The three Markdown files are the direct runtime knowledge source, while the PDFs provide supporting documentation through the frontend. This structure keeps the system understandable and maintainable while giving the AI a defined factual foundation and clear safety boundaries.